

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: CSA07 COURSE CODE: Computer Networks

COURSE OUTCOME COVERED

CO1: Apply the functions of OSI layer in enabling reliable data transmission across networks. (BL2, BL3)

Assessment Tool Weightage: 40%

Annexure A

SHORT ANSWER TEST

Dr.V.Parthipan

Code of Conduct:

I S. Karthikeyan / 192419048 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CSA07 — COMPUTER NETWORKS

Assessment Tool 1 — Short Answer Test: Worked Solutions

CO1: Apply the functions of OSI layers in enabling reliable data transmission across networks

Name: S. Karthikeyan | Guided by: Dr. V. Parthiban

Q1. File transfer — segments, frames, overhead & transmission time

A 150 MB file is sent over a 50 Mbps link. Transport Layer segments = 1500 bytes; Data Link Layer adds 40 bytes header+trailer per frame.

Given

File size = 150 MB = 150×10^6 bytes = 150,000,000 bytes

Segment size = 1500 bytes | Frame overhead = 40 bytes | Bandwidth = 50 Mbps

Step 1 — Number of segments (Transport Layer)

Segments = $150,000,000 \div 1500 = 100,000$ segments

Step 2 — Number of frames (Data Link Layer)

Each segment becomes one frame → Frames = 100,000 frames

Step 3 — Total Data Link overhead

Overhead = $100,000 \times 40$ bytes = 4,000,000 bytes = 4 MB

Step 4 — Total data actually transmitted

Total = $150,000,000 + 4,000,000 = 154,000,000$ bytes = 154 MB

Step 5 — Transmission time (ignoring propagation delay)

Total bits = $154,000,000 \times 8 = 1,232,000,000$ bits

Time = $1,232,000,000 \div (50 \times 10^6) = 24.64$ seconds

Explanation — Role of Transport & Data Link Layers

The Transport Layer breaks the file into fixed-size segments and adds sequence numbers, so segments can be reassembled correctly and lost or duplicate segments detected; protocols such as TCP use acknowledgements and retransmission to guarantee reliable end-to-end delivery. The Data Link Layer wraps each segment in a frame with header/trailer fields (e.g., addressing and a CRC checksum) that detect transmission errors on the physical link and trigger retransmission of damaged frames, ensuring reliable node-to-node delivery.

Q2. Sliding window — number of windows & retransmissions

Window size = 6 frames, each frame = 1024 bytes, sender transmits 48 frames; one frame lost per window.

Step 1 — Number of transmission windows

Windows = Total frames $\div$ Window size = $48 \div 6 = 8$ windows

Step 2 — Retransmissions

1 lost frame per window $\times$ 8 windows = 8 retransmissions

Explanation — How flow control improves efficiency

Sliding window flow control lets the sender transmit several frames before waiting for an acknowledgement, instead of stopping after every single frame (stop-and-wait). This keeps the link busy, improves throughput, and still lets the receiver control the pace of data so it is never overwhelmed. Only the lost frame within a window needs to be retransmitted, so bandwidth is not wasted resending frames that already arrived safely.

Q3. IP fragmentation — fragments & header overhead

A 12,000-byte IP packet passes through a router with MTU = 1500 bytes; each fragment adds a 20-byte IP header.

Given

Packet size = 12,000 bytes | MTU = 1500 bytes | Header per fragment = 20 bytes

Step 1 — Usable payload per fragment

Payload per fragment = MTU – header = 1500 – 20 = 1480 bytes

Step 2 — Number of fragments

Fragments = $\lceil 12,000 \div 1480 \rceil = \lceil 8.11 \rceil = 9$ fragments

(8 fragments of 1480 bytes = 11,840 bytes + 1 final fragment of 160 bytes)

Step 3 — Total header overhead

Overhead = 9×20 bytes = 180 bytes

Explanation — Role of the Network Layer

The Network Layer (IP) fragments a packet that is larger than a link's MTU into smaller pieces that each fit the link. Every fragment carries an identification field (to show which original packet it belongs to), a fragment offset (its position within the original data), and a more-fragments flag. The destination Network Layer uses these fields to reassemble the fragments back into the original packet in the correct order before passing it up to the Transport Layer, ensuring successful end-to-end delivery even across links with different MTUs.

Q4. Sliding window — number of windows & retransmissions

Window size = 6 frames, each frame = 1024 bytes, sender transmits 48 frames; one frame lost per window.

Step 1 — Number of transmission windows

Windows = Total frames $\div$ Window size = $48 \div 6 = 8$ windows

Step 2 — Retransmissions

1 lost frame per window $\times$ 8 windows = 8 retransmissions

Explanation — How flow control improves efficiency

Sliding window flow control lets the sender transmit several frames before waiting for an acknowledgement, instead of stopping after every single frame (stop-and-wait). This keeps the link busy, improves throughput, and still lets the receiver control the pace of data so it is never overwhelmed. Only the lost frame within a window needs to be retransmitted, so bandwidth is not wasted resending frames that already arrived safely.

Q5. Session checkpoints — recovery after failure

A 600 MB video is sent with a Session Layer checkpoint every 60 MB; failure occurs after 390 MB.

Step 1 — Checkpoints created before failure

Checkpoints occur at 60, 120, 180, 240, 300, 360 MB $\rightarrow$ 6 checkpoints reached before 390 MB

Step 2 — Data to be retransmitted after recovery

Data since the last checkpoint (360 MB) = $390 - 360 = 30$ MB must be retransmitted

Explanation — Importance of Session Layer synchronization

The Session Layer inserts synchronization checkpoints so that, if a session is interrupted, the transfer can resume from the last checkpoint instead of restarting from zero. This saves time and bandwidth, supports dialog control between the two communicating systems, and makes long transfers (like large video files) resilient to network failures.

Q6. Compression & encryption — final size and reduction

A 900 MB file is compressed by 40%, then encrypted with 5% overhead on the compressed size.

Step 1 — Compressed size

$$\text{Compressed size} = 900 \times (1 - 0.40) = 900 \times 0.60 = 540 \text{ MB}$$

Step 2 — Encrypted size

$$\text{Encrypted size} = 540 \times (1 + 0.05) = 540 \times 1.05 = 567 \text{ MB}$$

Step 3 — Overall percentage reduction vs. original

$$\text{Reduction} = (900 - 567) \div 900 \times 100 = 333 \div 900 \times 100 = 37\%$$

Explanation — Role of the Presentation Layer

The Presentation Layer improves transmission efficiency by compressing data, reducing the number of bytes that must travel across the network and cutting transmission time. It also improves security by encrypting the (compressed) data, protecting confidentiality even though encryption itself adds a small amount of overhead. Together, compression and encryption let the Presentation Layer trade a small overhead cost for large efficiency and security gains — here still a net 37% smaller than the original file.

Q7. FTP transfer — requests and transmission time

An FTP application transfers 3 GB over an effective throughput of 120 Mbps, with the Application Layer splitting data into 3 MB requests.

Step 1 — Number of requests

$$\text{Total data} = 3 \text{ GB} = 3000 \text{ MB}; \text{Requests} = 3000 \div 3 = 1000 \text{ requests}$$

Step 2 — Transmission time

$$\text{Total data in bits} = 3 \text{ GB} \times 8 = 3 \times 10^9 \times 8 = 24 \times 10^9 \text{ bits}$$

$$\text{Time} = 24 \times 10^9 \div (120 \times 10^6) = 200 \text{ seconds}$$

Explanation — How the Application Layer provides reliable service

The Application Layer (FTP) breaks a large transfer into manageable, individually-tracked requests, so any single failed request can be retried without resending the whole file. It also provides the user-facing service — authentication, file naming, directory browsing — and relies on the underlying Transport Layer (TCP) beneath it for guaranteed, in-order, error-checked delivery of each request's data.

Q8. End-to-end delay across OSI layers

Processing delays: Application = 4 ms, Transport = 3 ms, Network = 5 ms, Data Link = 2 ms, Physical = 1 ms.
Propagation delay = 18 ms, Transmission delay = 12 ms.

Step 1 — Total processing delay

$$4 + 3 + 5 + 2 + 1 = 15 \text{ ms}$$

Step 2 — Total end-to-end delay

$$\text{Total delay} = \text{Processing} + \text{Propagation} + \text{Transmission} = 15 + 18 + 12 = 45 \text{ ms}$$

Explanation — Contribution of each layer

Each OSI layer adds its own processing time to encapsulate or decapsulate the data (adding/removing headers, routing decisions, framing, error checks). On top of this per-layer processing, the packet also experiences transmission delay (time to push all the bits onto the link, determined by bandwidth) and propagation delay (time for a signal to physically travel the distance of the link, determined by medium and distance). The end-to-end delay is the sum of all of these contributions across every layer and every link the packet crosses.

Q9. Frame overhead and transmission efficiency

80 MB of useful data is sent using 1600-byte frames, each carrying 80 bytes of protocol overhead.

Given

Useful data = 80 MB = 80,000,000 bytes | Frame size = 1600 bytes | Overhead/frame = 80 bytes

Step 1 — Usable payload per frame

Payload per frame = 1600 – 80 = 1520 bytes

Step 2 — Total number of frames

Frames = $[80,000,000 \div 1520] \approx 52,632$ frames

Step 3 — Total overhead transmitted

Overhead = $52,632 \times 80$ bytes $\approx 4,210,560$ bytes ≈ 4.21 MB

Step 4 — Transmission efficiency

Efficiency = $\text{Payload} \div \text{Frame size} \times 100 = 1520 \div 1600 \times 100 = 95\%$

Explanation — Effect of protocol overhead

Every layer's header and trailer bytes are necessary for addressing, sequencing and error control, but they are not part of the actual message — they reduce the fraction of each frame that carries useful data. Here, 5% of every frame is overhead, so only 95% of the transmitted bits are useful payload. Higher overhead (smaller frames or more header fields) lowers efficiency and effectively wastes bandwidth, so protocol designers try to balance reliability features against this overhead cost.

Q10. Healthcare data transfer — combined layer processing

A 240 MB medical record is compressed 30% (Presentation Layer), split into 1400-byte segments (Transport Layer), and each frame gets 60 bytes of overhead (Data Link Layer).

Step 1 — Compressed file size

Compressed size = $240 \times (1 - 0.30) = 240 \times 0.70 = 168$ MB = 168,000,000 bytes

Step 2 — Number of segments (Transport Layer)

Segments = $168,000,000 \div 1400 = 120,000$ segments

Step 3 — Total Data Link overhead

Overhead = $120,000 \times 60$ bytes = 7,200,000 bytes = 7.2 MB

Step 4 — Final data transmitted across the Physical Layer

Total = $168,000,000 + 7,200,000 = 175,200,000$ bytes = 175.2 MB

Explanation — Multiple OSI layers working together

The Presentation Layer first compresses the sensitive record to reduce its size (and can encrypt it for confidentiality). The Transport Layer then splits the compressed data into fixed-size segments with sequence numbers, enabling reliable, in-order reassembly and retransmission of any lost segment. The Data Link Layer frames each segment with addressing and error-detection bytes to ensure it survives the local link. Together these layers let an IoT healthcare network deliver large, sensitive medical files efficiently, reliably, and securely to the destination.

Note: Unless a question specifies otherwise, MB/GB are taken as decimal units (1 MB = 10^6 bytes, 1 Mbps = 10^6 bits/sec), the standard convention in networking bandwidth/throughput problems.